

進階程式設計

Advanced Program Design

L7 Part II



Chia-Chi Chang

ccchang@cc.ncue.edu.tw

質數回顧

※除了1和自己本身之外，無法被其他數整除的數稱為質數

➤ 判斷1個整數是否為質數(窮舉法)

```
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>

int16_t checkIsPrime(int16_t p) {
    int16_t loop;

    for(loop=2; loop<p; loop++) {
        if(!(p % loop)) {
            return 0;
        }
    }
    return 1;
}

int main(int argc, char *argv[]) {
    int16_t p;

    while(scanf("%hd", &p) != EOF) {
        if(checkIsPrime(p)) {
            printf("%d is a prime number!\n",p);
        } else {
            printf("%d is not a prime number!\n",p);
        }
    }
    return 0;
}
```

檢查所有數字是否為質數

```
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>

int16_t checkIsPrime(int16_t p) {
    int16_t loop;

    for(loop=2; loop*loop <= p; loop++) {
        if(!(p % loop)) {
            return 0;
        }
    }
    return 1;
}

int main(int argc, char *argv[]) {
    int16_t p;

    while(scanf("%hd", &p) != EOF) {
        if(checkIsPrime(p)) {
            printf("%d is a prime number!\n",p);
        } else {
            printf("%d is not a prime number!\n",p);
        }
    }
    return 0;
}
```

只需檢查至 $\sqrt{p}$ 即可

判斷p是否為質數的策略

※先拿什麼數去除p，可以讓程式進一步優化

1) 判斷是否為2或3的倍數，這樣可以節省至少一半的時間

2) 觀察剩餘數的變化: 5, 7, 11, 13, 17, 19, 23, 25, 29, 31,...

➤ `int32_t isPrimeNumber(int32_t p)`用來檢查p是否為質數；回傳0代表不是質數，回傳1則代表質數

```
int32_t isPrimeNumber(int32_t p) {  
    if(p <= 1) return 0;  
    if(p == 2) return 1;  
    if(p == 3) return 1;  
  
    if(!(p % 2)) return 0;  
    if(!(p % 3)) return 0;  
  
    for(int32_t loop=5, gap=2; loop*loop<=p; loop+=gap, gap=6-gap) {  
        if(!(p % loop)) return 0;  
    }  
    return 1;  
}
```

C參考範例

查表法Lookup Table

- 質數已被證明有無限多個，因此題目通常會給定一個範圍，找出範圍內所有的質數
 - 若每次根據給定範圍再去找出所有質數，解題時間可能會超時。
- 根據輸入條件的最大限制，利用陣列預先建立一個質數表。
 - 乍看建立質數表會花費很多時間，剩下的也只是掃描陣列的動作(查表)。
- 建立質數表的缺點：花費記憶體空間大，有些時候需要另外的技巧去節省記憶體空間
 - 埃拉托斯特尼篩法(Eratosthenes)
 - 歐拉篩法(Euler's Sieve)

質數(Prime Number)與合數(Composite Number)

- 質數(Prime Number):除了1和自己之外，無法被其他自然數整除(除了0之外，非負整數)
- 合數(合併數)(Composite Number)：
 - 除了1和本身之外具有其他正因數的正整數
 - (ex. 16除了1和自己可以整除之外，還有2, 4, 8等因數可以整除16)
 - 大於1且不是質數可稱為合數
 - 2個或2個以上質因數的乘積(算術基本定理)
 - $6936 = 2^3 \times 3 \times 17^2$ ， $1200 = 2^4 \times 3 \times 5^2$ ， $5207 = 41 \times 127$

埃拉托斯特尼篩法(Eratosthenes)

※想法：標記1~n之間所有的合數，剩下未標記的數字即為質數

※作法：

- 1) 從最小質數 $p=2$ 開始，將 p 的所有倍數 $\{2*p, 3*p, 4*p, \dots, m*p \leq n\}$ 標記為合數。
- 2) 下一個未標記的最小自然數也將是質數($p'=3$)，遵循1)也將 p' 的所有倍數 $\{2*p', 3*p', \dots, p'*p', \dots, m'*p' \leq n\}$ 也標記為合數
- 3) 然後重複此一過程。

```
void eratosthenes(uint8_t prime[ARRAY_LENGTH]) {
    //Prime table is initialized
    for(int32_t loop=0; loop<ARRAY_LENGTH; loop++) {
        prime[loop] = 1;
    }
    prime[0] = 0;    //0 and 1 are labeled as non-prime numbers
    prime[1] = 0;

    for (int32_t iloop=2; iloop<ARRAY_LENGTH; iloop++) {
        if (prime[iloop]) {
            for (int32_t jloop=iloop+iloop; jloop<ARRAY_LENGTH; jloop+=iloop) {
                prime[jloop] = 0;    //prime[jloop] is labeled as a non-prime number
            }
        }
    }
}
```

C參考範例

埃拉托斯特尼篩法(Eratosthenes) – Cont'd

※優化1:在 $2 \sim \sqrt{n}$ 之間即可確認 n 是否為質數，當然也知道 n 之前所有的質數(亦也知道合數)

※優化2:在 $\{2*p, 3*p, \dots, p-1*p, p*p, \dots, m*p \leq n\}$ 之中，其 $2*p \sim p-1*p$ 已經被標註，可跳過以節省時間。

```
void eratosthenes(uint8_t prime[ARRAY_LENGTH]) {
    //Prime table is initialized
    for(int32_t loop=0; loop<ARRAY_LENGTH; loop++) {
        prime[loop] = 1;
    }
    prime[0] = 0;    //0 and 1 are labeled as non-prime numbers
    prime[1] = 0;

    for (int32_t iloop=2; iloop<ARRAY_LENGTH; iloop++) {
        if (prime[iloop]) {
            for (int32_t jloop=iloop+iloop; jloop<ARRAY_LENGTH; jloop+=iloop) {
                prime[jloop] = 0;    //prime[jloop] is labeled as a non-prime number
            }
        }
    }
}
```

C參考範例(未優化)

埃拉托斯特尼篩法(Eratosthenes) – Cont'd

※優化1:在 $2 \sim \sqrt{n}$ 之間即可確認 n 是否為質數，當然也知道 n 之前所有的質數(亦也知道合數)

※優化2:在 $\{2*p, 3*p, \dots, p-1*p, p*p, \dots, m*p \leq n\}$ 之中，其 $2*p \sim p-1*p$ 已經被標註，可跳過以節省時間。

❖ 是否還有其他優化的可能性？

```
void eratosthenes(uint8_t prime[ARRAY_LENGTH]) {
    //Prime table is initialized
    for(int32_t loop=0; loop<ARRAY_LENGTH; loop++) {
        prime[loop] = 1;
    }
    prime[0] = 0;    //0 and 1 are labeled as non-prime numbers
    prime[1] = 0;

    優化1
    for (int32_t iloop=2; iloop*iloop<ARRAY_LENGTH; iloop++) {
        if (prime[iloop]) {  優化2
            for (int32_t jloop=iloop*iloop; jloop<ARRAY_LENGTH; jloop+=iloop) {
                prime[jloop] = 0;    //prime[jloop] is labeled as a non-prime number
            }
        }
    }
}
```

C參考範例(優化)

手動增加Stack空間

- 程式需要宣告很大的記憶體空間時，程式編譯正常卻無法正常執行
- 記憶體空間超過Dev C++堆疊限制(預設2MB)，需手動增加堆疊空間

```
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>
#define ARRAY_LENGTH 30000000
void Init_Arr(int32_t arr[ARRAY_LENGTH]) {
    for(int32_t loop=0; loop<ARRAY_LENGTH; loop++) {
        arr[loop] = 255;
    }
}
int main(int argc, char *argv[]) {
    int32_t arr[ARRAY_LENGTH];
    printf("initialize the elements of arr to 255...\n");
    Init_Arr(arr);
    printf("end function Init_Arr ...\n");
    return 0;
}
```

Process exited after 1.984 seconds with return value 3221225725
Press any key to continue . . .

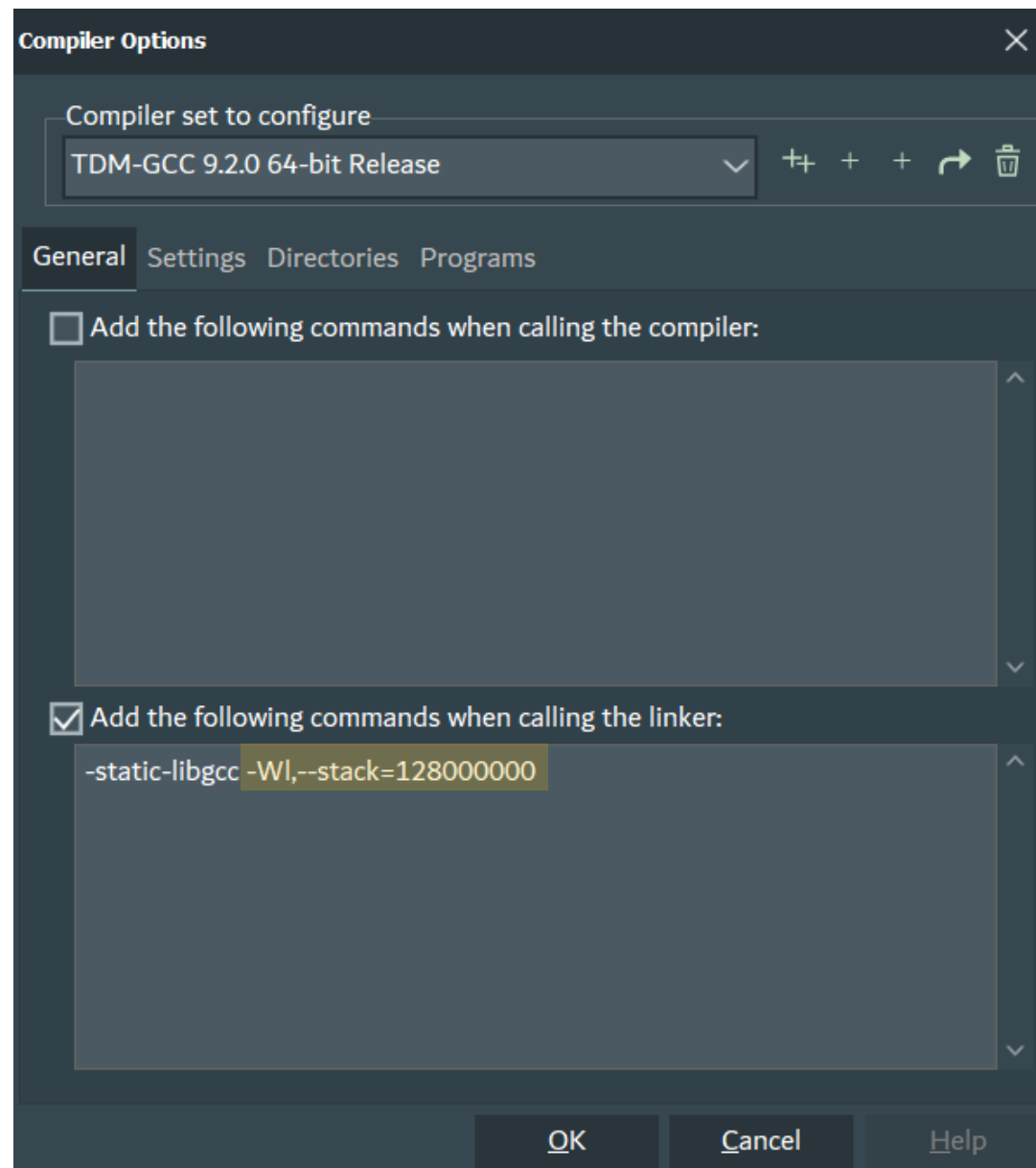
編譯成功卻輸出不正常

```
- Errors: 0
- Warnings: 0
- Output Filename: C:\Users\ccchang\Desktop\c\Project1.exe
- Output Size: 322.6298828125 KiB
- Compilation Time: 0.09s
```

Compile Log

手動增加Stack空間

- 於Compiler Options->General的Add the following commands when calling the linker的-static-libgcc之後，新增-Wl,--stack=記憶體空間(Bytes)(ex. -Wl,--stack=12800000)
- 重新編譯程式執行

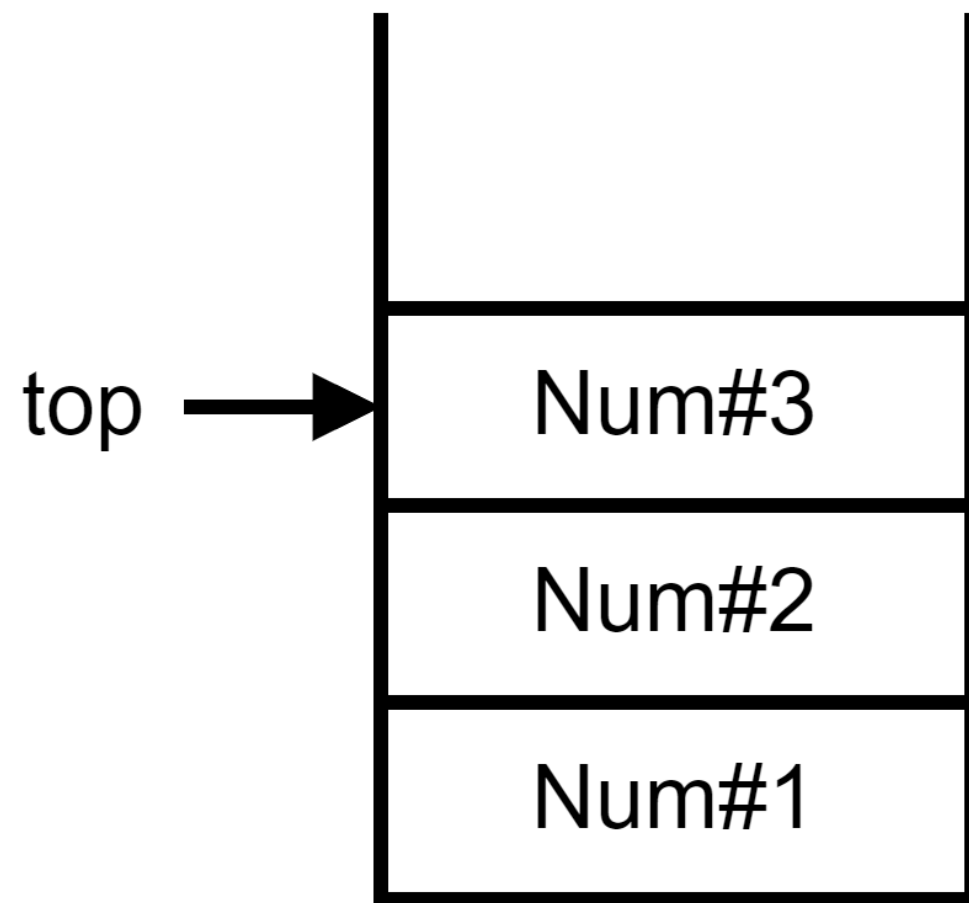


initialize the elements of arr to 255...
end function Init_Arr ...

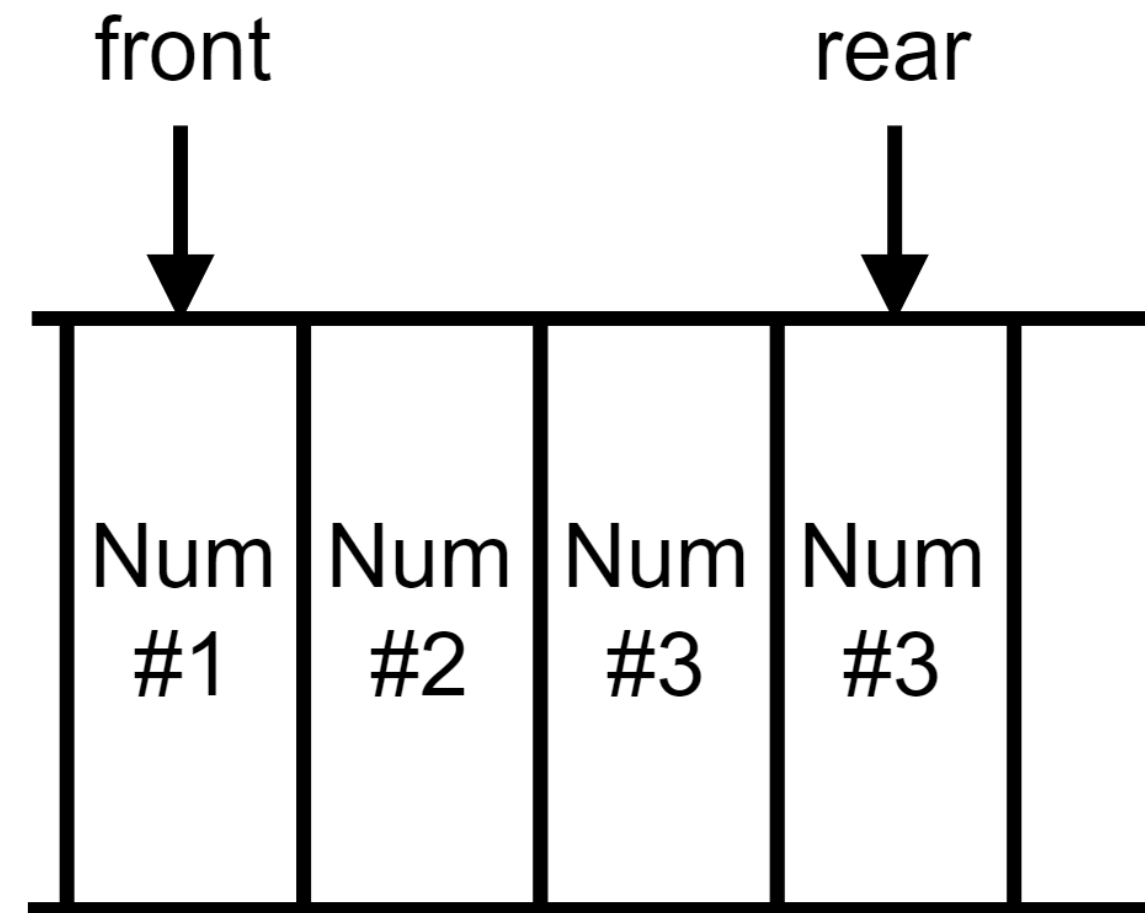
Process exited after 0.08548 seconds with return value 0
Press any key to continue . . .

堆疊(Stack)和佇列(Queue)

- 在演算法(Algorithm)中，堆疊(Stack)與佇列(Queue)是常用的資料結構
- 堆疊(Stack)屬於後進先出(Last In First Out, LIFO)線性串列，而且只能在頂端(top)進行資料的增加(Push)和刪除(Pop)。
- 佇列(Queue)屬於先進先出(First In First Out, FIFO)線性串列，新增資料發生在後端(rear)，而刪除資料在前端(front)。

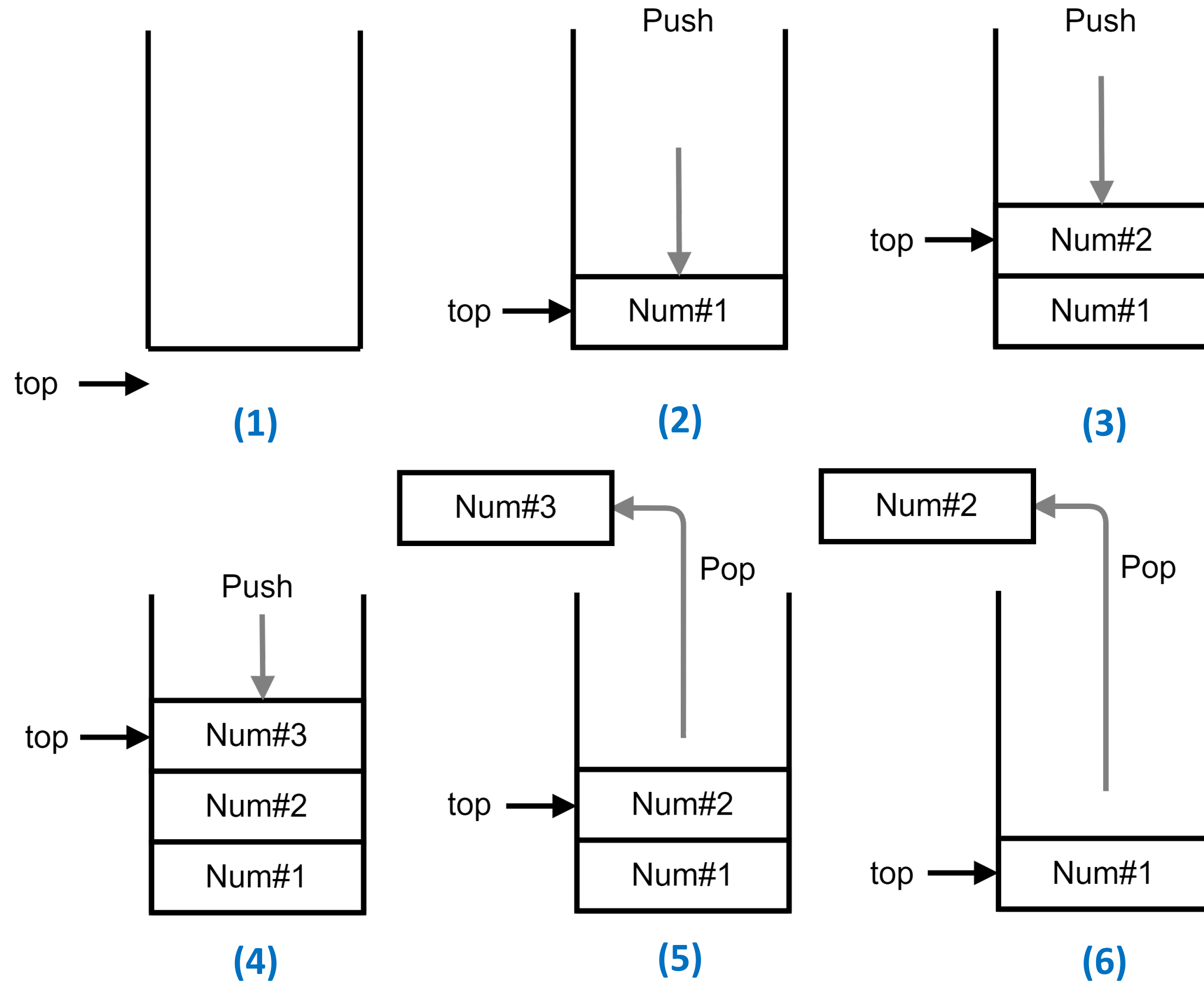


堆疊(Stack)

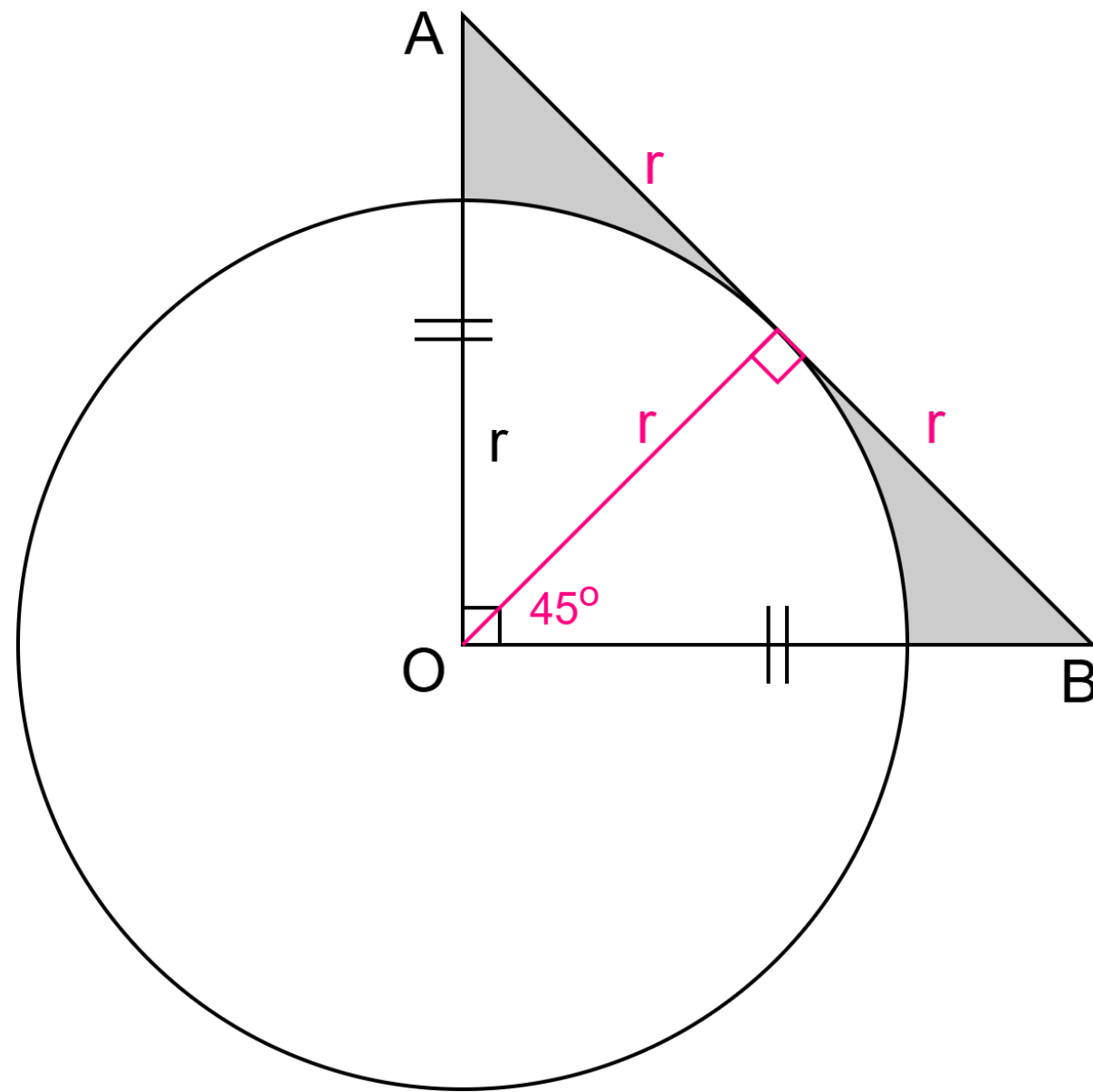


佇列(Queue)

堆疊(Stack)



Q6



陰影面積等於 $\triangle AOB - \frac{1}{4}$ 圓

$$= (2r * r * \frac{1}{2}) - (\frac{1}{4} * \pi * r^2)$$

$$= r^2 - 0.785 * r^2$$

$$= 0.215 * r^2$$

浮點運算有可能造成誤差

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <stdint.h>
int main(int argc, char *argv[]) {
    int16_t r;
    float farea;
    double darea;

    scanf("%hd", &r);
    farea = 0.215f * r * r * 100.0f;
    darea = 0.215 * r * r * 100.0;

    printf("farea = %.16f\n", farea);
    printf("darea = %.16f\n", darea);

    printf("%.2f\n", round(farea) / 100.0);
    printf("%.2f\n", round(darea) / 100.0);
    return 0;
}
```

使用電子計算機或Excel可得出

$0.215 * 19 * 19 = 77.615 \approx 77.62$

19

farea = 7761.5000000000000000

darea = 7761.4999999999991000

77.62

77.61

Process exited after 0.5343 seconds with return value 0
Press any key to continue . . .

方法1

```
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>
#include <math.h>
#define MY_EPSILON 1e-8
#define ARRAY_LENGTH 100
typedef struct {
    uint32_t r;
    double area;
} str_cc;
int main(int argc, char *argv[]) {
    int16_t k;
    str_cc arr[100];

    scanf("%hd", &k);
    for(uint8_t loop = 0; loop < k; loop++) {
        uint32_t r, a;
        scanf("%u", &r);
        arr[loop].r = r;
        double t1 = round(0.215 * r * r * 100.0);
        double t2 = round(0.215 * r * r * 100.0 + MY_EPSILON);
        if(abs(t1-t2) == 1) {
            arr[loop].area = t2 / 100.0;
        } else {
            arr[loop].area = t1 / 100.0;
        }
    }
    for(uint8_t loop = 0; loop < k; loop++) {
        printf("%.2f\n", arr[loop].area);
    }
    return 0;
}
```

方法2

```
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>
#include <math.h>
#define ARRAY_LENGTH 100
typedef struct {
    uint32_t r;
    uint32_t a;
} str_cc;
int main(int argc, char *argv[]) {
    int16_t k;
    str_cc arr[100];

    scanf("%hd", &k);
    for(uint8_t loop = 0; loop < k; loop++) {
        uint32_t r, a;
        scanf("%u", &r);
        arr[loop].r = r;
        a = 215 * r * r;
        if((a % 10) >= 5) {
            a -= a % 10;
            a += 10;
        } else {
            a -= a % 10;
        }
        arr[loop].a = a;
    }
    for(uint8_t loop = 0; loop < k; loop++) {
        printf("%.2f\n", arr[loop].a / 1000.0);
    }
    return 0;
}
```

The End

